DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L FULL STACK APPLICATION DEVELOPMENT

Name: SHAIK KHAJA AMEEN
ID:2400030234
Sec-3

SKILL - 3 ➔ Working with HQL - Sorting, Pagination & Aggregates

Prerequisites:

- **Basic knowledge of Hibernate**
- **Basic idea of HQL syntax**

Problem Statement
In this lab, we extend the Hibernate module created in Skill 2 and perform advanced data retrieval using HQL (Hibernate Query Language) including:

- **Sorting**
- **Pagination**
- **Filtering**
- **Aggregate functions**

These operations help in retrieving structured and meaningful data from the database.

1. Use the **Product entity** created in Skill 2.

```
package com.klu.model;

import javax.persistence.*;


@Entity
@Table(name="product")
public class Product {
   @Id
   @GeneratedValue(strategy=GenerationType.IDENTITY)
   int id;
   private String name;
    private  String description;
   private double price;
   private int quantity;

   public Product() {
```

```
    }
```

ID:2400030234

```
public void setName(String name ) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }

    public void setPrice(double price ) {
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public void setQuantity(int quantity ) {
        this.quantity = quantity;
    }

    public int getQuantity() {
        return quantity;
    }

}
```

2. Add **5–8 additional Product records** into the database.

3.Write HQL queries to retrieve all products sorted by price:

a. Ascending order

b. Descending order

ID:2400030234

```
private static void viewProductAsc() {
  Session session = factory.openSession();
  Query<Product> q = session.createQuery("from Product order by price", Product.class);
  System.out.println("\nName | Description | Price | Quantity");
  for (Product e : q.list()) {
  System.out.println(e.getName() + " | " + e.getDescription() + " | " +
  e.getPrice() + " | " + e.getQuantity());
  }
  session.close();
}
```

A.
```
private static void viewProductDesc() {
        Session session = factory.openSession();
        Query<Product> q = session.createQuery("from Product order by price desc",
    Product.class);
        System.out.println("\nName | Description | Price | Quantity");
        for (Product e : q.list()) {
        System.out.println(e.getName() + " | " + e.getDescription() + " | " +
        e.getPrice() + " | " + e.getQuantity());
        }
        session.close();
    }
```

4.Write an HQL query to sort products by quantity (highest first).

```
  private static void viewProductByQuantity() {
  Session session = factory.openSession();
  Query<Product> q = session.createQuery("from Product order by quantity desc",
Product.class);
  System.out.println("\nName | Description | Price | Quantity");
  for (Product e : q.list()) {
  System.out.println(e.getName() + " | " + e.getDescription() + " | " +
  e.getPrice() + " | " + e.getQuantity());
  }
  session.close();
}
```

5. Implement pagination using HQL to display:

c. First 3 products

d. Next 3 products

ID:2400030234

g. private static void getFirstThreeProducts() {

```
Session session = factory.openSession();
Query<Product> q = session.createQuery("from Product", Product.class);
q.setFirstResult(0);
q.setMaxResults(3);
System.out.println("\nName | Description | Price | Quantity");
for (Product e : q.list()) {
System.out.println(e.getName() + " | " + e.getDescription() + " | " +
e.getPrice() + " | " + e.getQuantity());
}
session.close();
}
```

h. private static void getNextThreeProducts() {

```
Session session = factory.openSession();
Query<Product> q = session.createQuery("from Product", Product.class);
q.setFirstResult(3);
q.setMaxResults(3);
System.out.println("\nName | Description | Price | Quantity");
for (Product e : q.list()) {
System.out.println(e.getName() + " | " + e.getDescription() + " | " +
e.getPrice() + " | " + e.getQuantity());
}
session.close();
}
```

6. Write HQL queries for aggregate operations:

e. Count total number of products

f. Count products where quantity > 0

g. Count products grouped by description

h. Find minimum and maximum price

```
private static void aggregateFunctions() {
Session session = factory.openSession();
Query q1 = session.createQuery("select count(*) from Product");
long totalProducts = (Long) q1.uniqueResult();

Query q2= session.createQuery("select count(*) from Product where quantity>0");
```

```
        long totalProducts1 = (Long) q2.uniqueResult();
```

ID:2400030234

```
        Query q3 = session.createQuery("select count(*) from Product group by description");
        long totalProducts2 = (Long) q3.uniqueResult();

        Query q4 = session.createQuery("select max(price) from Product");
        double maxPrice = (Double) q4.uniqueResult();

        Query q5 = session.createQuery("select min(price) from Product");
        Double minPrice = (Double) q5.uniqueResult();

         System.out.println("\nTotal Number of Products: "+totalProducts);
         System.out.println("Total Number of Products whose quantity is greater than zero:
   "+totalProducts1);
         System.out.println("Total Number of Products grouped by description:
   "+totalProducts2);
         System.out.println("Minimum Price: "+minPrice);
         System.out.println("Maximum Price: "+maxPrice);

   }
```

7. Write an HQL query using **GROUP BY** to group products by description.

```
private static void groupProductsByDesc() {
  Session session = factory.openSession();
  Query<Product> q = session.createQuery("from Product group by description",
Product.class);
  System.out.println("\nName | Description | Price | Quantity");
     for (Product e : q.list()) {
     System.out.println(e.getName() + " | " + e.getDescription() + " | " +
     e.getPrice() + " | " + e.getQuantity());
     }
     session.close();
}
```

8. Write an HQL query using **WHERE** to filter products within a price range.

```
     private static void getProductsWithinPriceRange() {
              Scanner sc = new Scanner(System.in);
                Session session = factory.openSession();
                System.out.print("\nEnter the minimum price: ");
                double mini=sc.nextDouble();
```

```
System.out.print("\nEnter the maximum price: ");
```

ID:2400030234

```
double maxi=sc.nextDouble();
        Query<Product> q = session.createQuery("from Product where price between
:min and :max", Product.class);
        q.setParameter("min",mini);
        q.setParameter("max",maxi);
        System.out.println("\nName | Description | Price | Quantity");
            for (Product e : q.list()) {
            System.out.println(e.getName() + " | " + e.getDescription() + " | " +
            e.getPrice() + " | " + e.getQuantity());
            }
            session.close();

    }
```

9. Write HQL queries using **LIKE** such as:

i. Names **starting with** certain letters

j. Names **ending with** certain letters

k. Names containing a pattern **anywhere** (substring)

l. Names with an **exact character length**

```
private static void productSearchByPattern() {
   Scanner sc = new Scanner(System.in);
     Session session = factory.openSession();
     System.out.print("\nEnter the product name pattern: ");
     String s = sc.nextLine();


     Query<Product> q = session.createQuery("FROM Product WHERE name LIKE
:name", Product.class);
     q.setParameter("name", s + "%");
     System.out.println("\nName | Description | Price | Quantity");
         for (Product e : q.list()) {
         System.out.println(e.getName() + " | " + e.getDescription() + " |
" +
         e.getPrice() + " | " + e.getQuantity());
         }
         session.close();

  }
```

10.Push the Hibernate project to GitHub.

https://github.com/Khaja-Ameen/GitFSADLab

ID:2400030234

OUTPUT

```
Console
MainApp (1) [Java Application] C:\Program Files\Java\jre1.8.0_461\bin\javaw.exe  (Feb 12, 2026, 8:03:58 PM elapsed: 0:00:25) [pid: 11408]

======Product Menu======

1. Insert Product
2. View Product details
3. Update Product details
4. Delete Product
5. View Product in Ascending Order By Price
6. View Product in Descending Order By Price
7. View Product in Descending Order By Quantity
8. Get the First Three Products
9. Get the next three Products
10. Perform aggregate functions
11. Group the products by description
12. Get the products within a price range
13. Get the product that starts with some pattern
14. exit
Enter Choice:
5

Name | Description | Price | Quantity
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name
Paint | Metallic Paint that prevents rust | 450.0 | 50
Gold | 18 Carat pure gold | 450.0 | 66
Shirt | Cotton Shirt | 560.0 | 22
Pant | Slim fit | 660.0 | 36
```

VIVA QUESTIONS (One-Line Answers)

1. What is HQL and how does it differ from SQL?
HQL is an object-oriented query language that works on entities instead of database tables.

2.Why do we use sorting in HQL?
Sorting is used to retrieve data in a specific ordered format.

3. What is pagination and where is it used?
Pagination limits the number of records displayed per page in large datasets.

4. Name any three aggregate functions supported in HQL.
COUNT, MIN, MAX (also SUM, AVG).

5. What is the difference between HQL and HCQL?
HQL is string-based query language, while HCQL (Criteria API) is programmatic query building.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation: